

RELATÓRIO: TRABALHO FINAL PROJETOS DIGITAIS

POR:

FABIO ALEIXO DE OLIVEIRA KARVAT GRR20254246

GABRIEL FELIPE RAUBER GRR20253456

LUCAS BARROSO LEAL GRR20255200

Para o trabalho, foi implementado um conjunto de instruções inspirado na arquitetura do intel 4004 para realizar um programa que calcula o n-ésimo número da sequência de fibonacci de forma interativa.

Seguem as instruções implementadas:

opcode	tipo	mnemônico	nome	operação
0000	I	beq	Branch equal	if (R[ra] = R[rb]) PC = PC + imm
0001	I	ji	Jump immediate	PC = PC + imm
0010	I	acc	accumulate	acc = rb
0011	X	X	X	X
0100	I	addi	Add immediate	R[rb] = R[ra] + imm
0101	X	X	X	X
0110	X	X	X	X
0111	X	X	X	X
1000	R	not	Not	R[rd] = ! R[ra]
1001	R	and	And	R[rd] = R[ra] && R[acc]
1010	R	or	Or	R[rd] = R[ra] R[acc]
1011	R	xor	Exclusive or	R[rd] = R[ra] ^ R[acc]
1100	R	add	Addition	R[rd] = R[ra] + R[acc]
1101	R	sub	Subtraction	R[rd] = R[ra] - R[acc]
1110	R	slr	Shift left register	R[rd] = R[ra] << R[acc]
1111	R	srr	shift right register	R[rd] = R[ra] >> R[acc]

INSTRUÇÃO TIPO I																
bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	opcode						imm				rb			ra		

INSTRUÇÃO TIPO R																
bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	opcode							rd			rb			ra		

As instruções são divididas nos seguintes campos:

opcode: indica qual a instrução a se realizar, é o utilizado pela unidade de controle.

imm: imediato, utilizado nas instruções tipo i.

rd: código do registrador destino, ou seja, o registrador onde o resultado de uma operação é escrito, seu sinal é ligado à entrada do banco de registradores que seleciona um registrador para ser escrito.

ra e rb: código dos registradores operandos, seus sinais são ligados às entradas do banco de registradores que selecionam registradores para serem lidos. O conteúdo desses registradores então ficam disponíveis nas saídas do banco de registradores para serem usados nas operações. ra normalmente é o operando principal enquanto rb é salvo no acumulador, rb também é o registrador destino para a instrução de addi.

Segue os códigos dos registradores:

1	registradores:
2	
3	x0 000
4	a0 001
5	t0 010
6	t1 011
7	t2 100
8	t3 101
9	t4 110

Operações:

implementação dos desvios: O circuito possui um registrador (PC) que indica a instrução a ser executada. A cada batida de clock ele recebe $PC + 1$ e passa para a próxima instrução. As instruções de desvio como `ji` enviam um sinal para um mux que faz PC receber $PC + imm$.

beq: pega os conteúdos de `ra` e `rb` e compara, se ($ra = rb$) faz o desvio.

addi: pega o conteúdo de `ra`, soma ao `imm` (utiliza um mux que determina se o segundo operando da ULA é `rb` ou o `imm`) e escreve em `rd`.

acc: guarda o valor de `rb` para ser usado como segundo operando.

operações ULA: pega os conteúdos de `ra` e `rb`, faz a operação na ULA e escreve o resultado em `rd`.

Componentes:

banco de registradores: um banco com 8 registradores que guardam valores de 4 bits, um deles (`x0`) tem valor zero fixo e não pode ser escrito, 7 deles são usados no cálculo do fibonacci.

ULA: uma ULA de 4 bits que faz as 8 operações listadas inicialmente.

unidade de controle: recebe o opcode e ativa determinados sinais de acordo com a demanda da instrução.

Sinais da unidade de controle:

PC_sel: sinal do mux que faz os desvios.

rd: sinal do mux que indica se o registrador destino é `rd` ou `rb` (`rb` é usado no `addi`).

wbreg: sinal que indica se o banco de registradores vai escrever um valor novo em `rd`.

muxula: sinal do mux que indica se o segundo operando da ULA é `rb` ou `imm`.

OpULA: sinal do mux que indica a operação da ULA.

acc: desvia a saída do banco de registradores para o acumulador, também determina se a entrada é `ra` (para operações) ou `rb` (para entrar no acumulador).

Segue o código do fibonacci iterativo utilizado:

```
addi a0, x0, 7
addi t0, x0, 0
addi t1, x0, 1
addi t2, x0, 1
#loop
acc x0
beq a0, acc, 9
acc t0
add t2, t1, acc
acc t1
add t0, x0, acc
acc t2
add t1, x0, acc
addi a0, a0, -1
ji -9
#fim
acc t2
add a0, x0, acc
```